

Architecture & Integration Strategy Report

AI-Powered Offline Sales Conversation Intelligence Platform

Document Ref: SYS-ARCH-02 (supersedes SYS-ARCH-01) • Version 2.0 • Prepared by: Akul Ranjan

Classification: Restricted Distribution

This report details the end-to-end technical blueprint for a scalable, AI-powered Offline Sales Conversation Intelligence Platform. The system securely converts code-mixed offline sales audio into structured, actionable business intelligence. Version 2.0 expands the original blueprint with an **integration contract and guardrail layer** — webhook security, delivery guarantees, idempotency, schema versioning, deprecation governance, tenant-scoped credentials, and a compliance posture suitable for enterprise procurement — reflecting current industry best practice for building software that plugs cleanly into a customer's existing stack.

1. Core System Architecture

The system is built on a heavily decoupled, asynchronous microservices architecture. Large media uploads and intensive LLM inferencing never block the core application servers or degrade frontend experience. Internally, durable message queues provide guaranteed delivery, retry, and ordering between services; externally, the platform speaks standard HTTPS webhooks and REST — the correct division of labour, since queues excel at internal service-to-service reliability while webhooks remain the universal external integration primitive.

1.1 Edge Ingestion & API Gateway

Audio is captured via mobile or web clients. Rather than routing large payloads through the backend, the API issues an ephemeral AWS S3 Presigned URL and clients upload directly to private cloud storage. On completion, a lightweight metadata webhook registers the job with the FastAPI backend, which enqueues it in Redis. The gateway also enforces per-tenant authentication, request validation, and rate limiting (Section 5.4) as the single policy enforcement point.

1.2 Asynchronous Processing Pipeline

Dedicated Celery workers poll the Redis broker. On dequeue, a worker dispatches the S3 audio URI to the AI Transcription API (e.g., Sarvam Saaras V3) with speaker diarization enabled, then terminates to free compute. Each job carries a unique **job_id** used as an idempotency key end-to-end, so a re-delivered trigger can never produce a duplicate transcription charge or duplicate downstream insight.

1.3 LLM Insight Extraction Layer

On transcription completion, an inbound webhook (signature-verified per Section 4.2) triggers the insight phase. The diarized transcript is piped into a large language model (e.g., a 105B-parameter model) using strict JSON-mode prompting against a versioned output schema. Extracted fields include customer sentiment, buying intent, specific objections, and agent compliance scores. LLM output is validated against the schema before persistence; malformed generations are retried with bounded backoff and quarantined to a dead-letter queue on repeated failure rather than silently dropped.

1.4 Outbound Event Bus (new in v2.0)

All externally visible state changes (call ingested, transcription complete, insights ready, export delivered) are published as canonical events to an internal outbox table and fanned out by a dedicated dispatcher service. This transactional-outbox pattern guarantees that an event is emitted if and only if the underlying database write committed — eliminating the classic dual-write inconsistency between the database and external consumers. The dispatcher, not the business services, owns retries, signing, and delivery bookkeeping, keeping integration concerns out of core domain code.

2. Enterprise Database Schema (PostgreSQL)

The relational schema is designed for strict multi-tenancy and high-speed analytical reads. Version 2.0 adds integration bookkeeping tables (WebhookEndpoints, WebhookDeliveries, ApiCredentials) so that delivery state, replay, and credential scoping are first-class data, not log archaeology.

Table	Primary Function	Critical Columns
Organizations	Top-level B2B tenants.	id, name, plan, data_region
Users	Platform authentication & roles.	id, org_id, role (Agent/Admin)
Calls	Central interaction entity.	id, org_id, agent_id, s3_url, status, client_metadata (JSONB)
Transcripts	Time-stamped text segments.	id, call_id, speaker_label, text_segment, start_ms, end_ms
Insights	Structured AI evaluation metrics.	call_id, schema_version, sentiment_score, key_objections (Array), compliance_score
ApiCredentials	Scoped, rotatable API keys per tenant.	id, org_id, key_hash, scopes (Array), expires_at, last_used_at
WebhookEndpoints	Customer-registered consumer URLs.	id, org_id, url, secret_hash, event_types (Array), status, api_version
WebhookDeliveries	Per-event delivery ledger.	id, endpoint_id, event_id, attempt, status, response_code, next_retry_at

Every tenant-owned table carries `org_id` and is protected by Row-Level Security (Section 5.1). The single `api_key_hash` column from v1.0 is deliberately replaced by the `ApiCredentials` table: enterprise consumers require multiple least-privilege keys (e.g., read-only analytics vs. webhook management), independent rotation, and audit trails per credential.

3. The Integration Contract (new in v2.0)

Industry practice is unambiguous: integratable platforms do not build point-to-point connectors first — they publish a stable, versioned, self-describing contract and let the ecosystem adapt to it. The contract below is the foundation on which all three rollout phases (Section 6) sit.

3.1 Thin Events + Rich APIs (Hybrid Pattern)

Mature integrations are hybrid: webhooks/events for triggering, REST APIs for enrichment, validation, and reconciliation. Outbound webhooks therefore carry a **thin payload** — `event_id`, `event_type`, `occurred_at`, `org_id`, resource type and ID, `schema_version` — plus a fetch URL. Consumers retrieve the full insight document via the authenticated REST API. This keeps PII out of intermediary pipes (Zapier, Make.com), shrinks the blast radius of a leaked payload, and lets the API response evolve independently of the event envelope.

3.2 Canonical Event Envelope

Field	Purpose
<code>event_id</code> (UUIDv7)	Globally unique; the consumer's idempotency/deduplication key.
<code>event_type</code>	Namespaced, e.g. <code>call.insights.ready</code> , <code>call.transcription.completed</code> .
<code>schema_version</code>	Explicit payload contract version, e.g. "1.0". Present from day one.
<code>occurred_at</code> / <code>delivered_at</code>	Event time vs. delivery time; supports ordering checks and replay-window validation.
<code>org_id</code>	Tenant scoping; consumers must verify it matches their own tenancy.

Field	Purpose
data	Thin resource reference: {resource: "call", id, status, fetch_url}.
sequence	Monotonic per-resource counter so stale events cannot overwrite newer state.

3.3 Versioning & Deprecation Governance

Once a single customer wires the JSON output into their CRM, the schema is a contract. The platform adopts URI-path versioning for the REST API (/api/v1/...) and an explicit schema_version field in every event, with the following non-negotiable rules:

- **Additive-only within a major version.** New optional fields and new endpoints are permitted; removing fields, renaming attributes, changing types, or making an optional parameter required forces a new major version.
- **Contract tests in CI.** Every change runs automated schema-compatibility checks (OpenAPI diff / consumer contract tests) so a breaking change cannot ship unnoticed.
- **Deprecation policy.** Major versions are announced 6–12 months before sunset, with changelogs, migration guides, and per-tenant usage telemetry to confirm migration before shutdown.
- **Governance ownership.** A named API owner reviews all contract changes; major-version proposals require an impact assessment covering affected tenants and compliance implications.

4. Webhook Delivery Guardrails (new in v2.0)

4.1 Outbound Authentication: HMAC + Timestamp + Replay Defence

Every outbound webhook is signed with HMAC-SHA256 over the raw request body using a per-endpoint secret, delivered in an X-Signature header alongside X-Timestamp. Consumers verify the signature against the raw bytes, reject requests whose timestamp falls outside a ± 5 -minute window, and deduplicate on event_id. Together these defeat the canonical webhook attack surface: forged requests, tampering in transit, and replay of captured deliveries. All traffic is HTTPS-only; per-endpoint secrets are rotatable without downtime via dual-secret overlap windows.

4.2 Inbound Webhook Verification

The same discipline applies in reverse. Callbacks from the transcription provider and cloud-messaging APIs are verified (provider signature, timestamp freshness) **before** the payload touches business logic, acknowledged with HTTP 200 immediately, and processed asynchronously off a queue — synchronous processing risks provider timeouts, which trigger provider retries and cascade into duplicate processing and race conditions.

4.3 At-Least-Once Delivery, Retries & Dead-Letter Queue

Webhook delivery is at-least-once by nature: duplicates and out-of-order arrival are inevitable and must be designed for, not hoped against. The dispatcher retries failed deliveries with exponential backoff and jitter (e.g., 1m $\rightarrow$ 5m $\rightarrow$ 30m $\rightarrow$ 2h $\rightarrow$ 12h, max 5 attempts). Exhausted deliveries land in a dead-letter queue visible to both operators and the customer's admin console, with one-click replay once the consumer endpoint recovers. Endpoints failing persistently are auto-disabled with notification rather than left to absorb silent data loss.

4.4 Idempotency & Ordering on Both Sides

Internally, every processing stage records processed event/job IDs so a redelivered message short-circuits without side effects. Externally, documentation and SDK examples instruct consumers to (a) store processed event_ids and skip duplicates, and (b) use the per-resource sequence number so a delayed older event can never overwrite newer state — idempotency alone is insufficient if stale events can clobber valid data.

OPERATING RULE No webhook endpoint goes live for any customer until HMAC signing, timestamp validation, event_id deduplication, retry-with-backoff, and DLQ capture are all in place. Retrofitting these onto live integrations is an order of magnitude more expensive than shipping them first.

4.5 Observability

The WebhookDeliveries ledger powers delivery dashboards: success rate, p95 consumer latency, retry rate, and DLQ depth per endpoint, with structured logs carrying org_id and event_id for distributed tracing. Slow or flapping consumer endpoints are surfaced to the customer proactively — delivery health is treated as a shared, visible SLO rather than a support-ticket mystery.

5. Security & Compliance Protocols

5.1 Row-Level Security (RLS)

The database layer enforces policies guaranteeing that queries only return rows matching the authenticated tenant_id, structurally eliminating cross-tenant leakage. RLS extends to the new integration tables: a tenant can only register, inspect, or replay deliveries for its own endpoints.

5.2 Ephemeral Data Access

S3 storage is fully private. Playback and access are governed exclusively by short-lived Presigned URLs generated by the backend per authorised request, never embedded in webhook payloads or third-party automation flows.

5.3 Edge Audio Compression

To survive uplink failures in high-noise, low-bandwidth field environments, mobile clients locally compress audio to 16 kHz mono prior to upload, with resumable multi-part transfer for unstable networks.

5.4 Scoped Credentials & Rate Limiting (new in v2.0)

- **Least-privilege API keys.** Tenants mint multiple keys with explicit scopes (insights:read, webhooks:manage, exports:create), independent expiry, and rotation; every call is attributable to a specific credential.
- **Per-tenant rate limits and quotas** enforced at the gateway, with standard 429 + Retry-After semantics, protecting shared infrastructure from a single noisy integration.
- **Secret hygiene.** Webhook secrets and key hashes are stored hashed; raw secrets are shown once at creation and rotatable via overlapping dual-secret windows.

5.5 Regulatory & Procurement Posture (new in v2.0)

Recorded sales conversations are PII-dense and jurisdictionally sensitive. The platform roadmap treats the following as enterprise table stakes: consent-to-record capture and storage per call; configurable retention and verified deletion of audio, transcripts, and derived insights; data-residency pinning via the Organizations.data_region column (notably for India's DPDP Act and GDPR contexts); SOC 2-aligned controls and audit logging; and contractual sub-processor disclosure for the transcription and LLM providers. These artefacts are prepared ahead of Phase 3, because enterprise security review — not engineering — is typically the longest pole in large deployments.

6. Phased “Plug & Play” Integration Strategy

Rather than developing brittle point-to-point connections with every conceivable CRM, the platform employs an outward-facing, contract-first architecture rolled out in phases. Each phase consumes the same canonical events and APIs defined in Section 3 — no phase forks the core.

Phase 1 — The “Shadow CRM” (Cloud Messaging)

Recognising that substantial offline sales coordination happens on messaging apps, the system integrates with standard Cloud Messaging APIs (e.g., WhatsApp Business). Once insights are extracted, structured summaries are pushed autonomously to managers' devices, creating an immediate feedback loop with zero IT friction. Summaries link back into the platform rather than embedding sensitive transcript content in the message channel.

Phase 2 — Middleware Ecosystem (Webhooks + REST API)

For mid-market customers on varied CRMs (HubSpot, LeadSquared), the platform exposes the signed outbound webhooks and versioned REST API from Sections 3–4. Customers route events through automation layers such as Zapier or Make.com — e.g., automatically appending AI sentiment scores to a lead's CRM profile — without engineering involvement on either side. A self-serve developer portal provides endpoint registration, secret rotation, delivery logs with replay, sandbox keys, and webhook test events, since discoverable tooling is what converts an API into an ecosystem.

Phase 2.5 — Native Connectors via Embedded iPaaS (new in v2.0)

When customers begin demanding in-product, one-click CRM connections (rather than self-assembled Zapier flows), the platform embeds an integration layer (embedded iPaaS or unified-API provider) instead of hand-building connectors. These platforms absorb the hidden majority of connector cost — OAuth token refresh race conditions, third-party rate limits, credential vaulting, schema drift, and API deprecations across hundreds of SaaS targets — while the customer experiences a fully native “Connect HubSpot” button inside our UI. Build-vs-embed is re-evaluated per connector: deep, revenue-critical integrations may justify first-party builds; the long tail should not.

Phase 3 — Custom Enterprise Adapters

For large organisational deployments (e.g., legacy ERPs), dedicated mapping adapters intercept the platform's standardised JSON and translate it into the proprietary enterprise schema. Adapters live outside the core, consume only the public contract, and are pinned to an explicit API major version — ensuring the core platform remains clean, decoupled, and independently evolvable. Enterprise rollouts additionally bundle the compliance artefacts from Section 5.5 and bespoke SLAs on webhook delivery.

Phase 4 — Agentic Access via MCP (forward-looking)

Integration consumers are increasingly AI agents rather than humans clicking UIs. Exposing the insights API as a Model Context Protocol (MCP) server would let customers' AI copilots query call intelligence directly (“what objections did the Ahmedabad team hear most this week?”) through a single governed endpoint — reusing the same scoped credentials, rate limits, and RLS guarantees as every other consumer.

7. Integration Readiness Checklist

#	Guardrail	Status target
1	HMAC-SHA256 signing + timestamp on all outbound webhooks; verification on all inbound webhooks.	Before first external consumer
2	event_id (UUIDv7) + consumer dedup guidance; per-resource sequence numbers against stale overwrites.	Before first external consumer
3	schema_version in every payload; /api/v1 path versioning; additive-only change policy with CI contract tests.	Day one
4	Retry with exponential backoff + jitter; dead-letter queue with operator and customer-visible replay.	Phase 2 launch
5	Scoped, rotatable API credentials (ApiCredentials table); per-tenant rate limits with 429/Retry-After.	Phase 2 launch

#	Guardrail	Status target
6	Delivery observability: success rate, retry rate, DLQ depth, p95 latency per endpoint; auto-disable flapping endpoints.	Phase 2 launch
7	Developer portal: endpoint registration, secret rotation, sandbox keys, test events, delivery logs, changelog.	Phase 2 maturity
8	Deprecation governance: 6–12 month sunset notices, migration guides, per-tenant version telemetry.	Before first v2 contract change
9	Compliance pack: consent capture, retention/deletion, data residency, SOC 2-aligned controls, sub-processor list.	Before Phase 3 procurement
10	Build-vs-embed decision per connector; embedded iPaaS evaluation for the long tail.	Phase 2.5 gate

Revision history — v1.0 (SYS-ARCH-01): core architecture, schema, three-phase integration strategy, baseline security. v2.0 (SYS-ARCH-02): added outbound event bus and transactional outbox; canonical event envelope; webhook signing, replay defence, retries, idempotency and DLQ; API/schema versioning and deprecation governance; scoped credentials and rate limiting; compliance posture; Phase 2.5 (embedded iPaaS) and Phase 4 (MCP/agentic access); integration readiness checklist.